



Санкт-Петербургский государственный университет
Кафедра системного программирования

Динамическая бинарная трансляция в RISC-V с помощью Instrew

Михайлов Илья Игоревич, группа 21.Б07-мм

Научный руководитель: к.ф.-м.н. Д.В. Луцив, доцент кафедры системного программирования

Консультант: В.А. Кутуев, инженер-исследователь, Лаборатория технологий
программирования инфраструктурных решений СПбГУ

Санкт-Петербург
2025

- Бинарные трансляторы являются инструментами для анализа кода, профилиции и эмуляции
- Важным классом которых являются динамические бинарные трансляторы (далее — ДБТ)
- В качестве промежуточного представления для ДБТ хорошо использовать LLVM IR
 - ▶ Тулчейн LLVM позволяет проводить качественные оптимизации
- Хотим ДБТ для RISC-V

- Динамические бинарные трансляторы:
 - ▶ QEMU-user
 - ▶ Rosetta 2
 - ▶ Valgrind
 - ▶ Box64
- Динамические бинарные трансляторы с поднятием в LLVM IR:
 - ▶ Instrew + Rellume
 - ▶ BinRec, HQEMU (не обновлялись с 2022 и 2018 года соответственно)
 - ▶ DBILL, LnQ, Risotto (есть только статьи, либо код закрыт)

- Выводы:

- ▶ Instrew производительнее, чем QEMU и Valgrind (основываясь на бенчмарках¹)
- ▶ Instrew "из коробки" имеет API для добавления инструментов бинарного анализа
- ▶ Из всех рассмотренных ДБТ с поднятием в LLVM IR Instrew + Rellume поддерживается мейнтейнером
- ▶ Instrew уже имеет guest-поддержку RISC-V

¹<https://mediatum.ub.tum.de/doc/1614897/1614897.pdf>

Постановка задачи

Цель: Добавить поддержку host-архитектуры RISC-V для ДБТ Instrew

Задачи:

- 1 Протестировать на произвольных статически слинкованных программах
- 2 Исправить ошибки, выявленные при тестировании
- 3 Добавить в CI поддержку сборки на RISC-V

Тестирование (1/2)

- Для выявления ошибок и улучшения поддержки RISC-V-хоста
- Используем следующие тест-кейсы:
 - ▶ статически и динамически слинкованные программы из coreutils под:
 - ★ x86_64
 - ★ riscv64
- В процессе тестирования было сделано:
 - ▶ Добавлены новые релокации и обновлена PLT-таблица
 - ▶ Сделан рефакторинг
- Из трудностей можно выделить:
 - ▶ Частично тестирование проводилось на плате Sipeed Lichee PI 4A, где был установлен специфический glibc, в следствие чего было потрачено время на поиск плохо задокументированных инструкций, которые Rellume не мог поднять в LLVM IR

Тестирование (2/2)

- В процессе тестирования оказалось, что в Rellume плохая поддержка RISC-V-хоста
 - ▶ Некоторые тест-кейсы используют MCJIT, которые падают с SEGFAULT на RISC-V
- Хотим попробовать обновить версию LLVM до 19, что потенциально может решить проблему
 - ▶ Делаем рефакторинг
 - ▶ Оказывается, что при обновлении дополнительно перестает работать MCAssembler, который также падает с SEGFAULT, тем самым уменьшая количество успешно пройденных тестов

- Есть трудности в связи с отсутствием нативной поддержки RISC-V в GH Actions
- Есть два варианта решения этой проблемы:
 - ▶ с помощью self-hosted runner на RISC-V
 - ▶ с помощью qemu-user и chroot
- Удалось опробовать оба решения, но был выбран второй подход, так как:
 - ▶ есть сложности с поддержкой self-hosted runner на RISC-V
 - ▶ удалось найти third-party action, с помощью которого не нужно настраивать qemu-user и chroot
- Также после добавления CI был сделан PR в основную ветку Instrew

В результате работы были выполнены следующие задачи:

- 1 Instrew был протестирован на произвольных статически слинкованных программах
- 2 Исправлены некоторые ошибки, выявленные при тестировании
- 3 В CI добавлена поддержка сборки на RISC-V

Код доступен по данной ссылке:

<https://github.com/aengelke/instrew/pull/20>.

```

0000000002747c <abort@GLIBC_2.27>:
2747c: 7131      addi    sp,sp,-192
2747e: f922      sd      s0,176(sp)
27480: 0014d417 auipc   s0,0x14d
27484: 47040413 addi    s0,s0,1136 # 1748f0 <__fpu_contr
ol@GLIBC_2.27+0x88>
27488: 641c      ld      a5,8(s0)
2748a: 0014c717 auipc   a4,0x14c
2748e: a9e73703 ld      a4,-1522(a4) # 172e98 <__stack_c
hk_guard@GLIBC_2.27>
27492: f926      sd      s1,168(sp)
27494: f14a      sd      s2,160(sp)
27496: fd06      sd      ra,184(sp)
27498: 6314      ld      a3,0(a4)
2749a: ed36      sd      a3,152(sp)
2749c: 4681      ll      a3,0
2749e: 8a020913 addi    s2,fp,-1888 # ffffffff8a0 <
__key_encryptsession_pk_LOCAL@GLIBC_2.27+0xffffffffff7ee78>
274a2: 84a2      mv      s1,s0
274a4: 0f278a63 beq     a5,s2,27598 <abort@GLIBC_2.27+0
x11c>
274a8: 4705      li      a4,1
274aa: 01a0000b lwr.w   a5,(s0)
274ac: 0520006f j       27500 <abort@GLIBC_2.27+0x84>
274b2: 00000013 nop
274b6: 00000013 nop
274ba: 00000013 nop
274be: 00000013 nop
274c2: 00000013 nop
274c6: 00000013 nop
274ca: 00000013 nop
274ce: 00000013 nop
274d2: 00000013 nop
274d6: 00000013 nop
274da: 00000013 nop
274de: 00000013 nop
274e2: 00000013 nop
274e6: 00000013 nop
274ea: 00000013 nop

000000000274b8 <abort@GLIBC_2.27>:
274b8: 7131      addi    sp,sp,-192
274ba: f922      sd      s0,176(sp)
274bc: 00133417 auipc   s0,0x133
274c0: 43440413 addi    s0,s0,1076 # 15a8f0 <__fpu_c
ontrol@GLIBC_2.27+0x88>
274c4: 641c      ld      a5,8(s0)
274c6: 00132717 auipc   a4,0x132
274ca: 9d273703 ld      a4,-1582(a4) # 158e98 <__sta
ck_chk_guard@GLIBC_2.27>
274cc: f926      sd      s1,168(sp)
274d0: f14a      sd      s2,160(sp)
274d2: fd06      sd      ra,184(sp)
274d4: 6314      ld      a3,0(a4)
274d6: ed36      sd      a3,152(sp)
274d8: 4681      ll      a3,0
274da: 8a020913 addi    s2,fp,-1888 # ffffffff8a0 <
a0 <__key_encryptsession_pk_LOCAL@GLIBC_2.27+0xffffffffff7ee78>
274de: 84a2      mv      s1,s0
274e0: 03278863 beq     a5,s2,27500 <abort@GLIBC_2.
27+0x48>
274e4: 4705      li      a4,1
274e6: 1a0427af lwr.w   a5,(s0)
274ea: c781      bnez   a5,274f2 <abort@GLIBC_2.27+
0x3a>
274ec: 18e426af sc.w    a3,a4,(s0)
274f0: fafd      bnez   a3,274e6 <abort@GLIBC_2.27+
0x2c>
274f2: 2781      sext.w a5,a5
274f4: c781      beqz   a5,274fc <abort@GLIBC_2.27+
0x44>
274f6: 8522      mv      a0,s0
274f8: 77b438ef jal     6b472 <__l1l_lock_wait_priva
te@GLIBC_PRIVATE>
274fc: 01243423 sd      s2,8(s0)
27500: 00133797 auipc   a5,0x133
27504: 3f47a783 lw      a5,1012(a5) # 15a8f4 <__fpu_
control@GLIBC_2.27+0x8c>
27508: 00133717 auipc   a4,0x133

```